The Cost of Observability: A Simulation Study of How Trace Sampling, Scrape Intervals, Cardinality Caps and Log Sampling Delay or Prevent Incident Detection

Pujitha Sri Lakshmi Paladugu
Independent Researcher, Lynnwood, Washington, USA
pujitha.rathna@gmail.com · ORCID 0009-0001-5678-4022

Running title: Cost of observability versus detection

Acknowledgments. Not applicable.

Abstract

Operators of cloud services reduce telemetry volume to control monitoring cost: they sample traces, lengthen metric scrape intervals, cap label cardinality and sample logs. Network management has long studied the accuracy–overhead trade-off of polling and packet sampling, but the effect of these service-level reductions on incident detection is rarely measured. We present a reproducible, seeded discrete-time simulation of a 30-service, 59-endpoint microservice system with heavy-tailed request rates and 2,000 Zipf-distributed tenants. We inject five incident classes: a global error spike, a low-traffic endpoint error, a 99th-percentile latency regression on one dependency, a gradual error creep and a single-tenant failure. Forty-three pipeline configurations and 1,247 combinations are evaluated under identical alert rules over 2,500 incident trials and 10 incident-free days, and volumes are priced with three public list-price books. Head sampling at 1% detected 4.8% of low-traffic endpoint failures, against 89.8% at full fidelity. Tail sampling matched full-fidelity detection in every class with 3.9% of the span volume and 25–35 s of added delay, provided its buffer had headroom above peak load. Severity-aware log sampling at 1% preserved error detection but not latency detection. Scrape intervals up to 120 s added at most 40 s of median delay. Capping the tenant label at 100 values reduced tenant-failure detection from 100% to 44%. Closed-form models of capture probability, scrape delay and cardinality dilution agree with the simulation. We map every pipeline to OpenTelemetry Collector and Prometheus configuration and give a procedure for choosing reduction settings.

Keywords: 4. Functional areas: Performance management · 7. Methods: Network monitoring and measurements · 7. Methods: Computer simulation experiments · Distributed tracing · Tail-based sampling · Monitoring cost

1 Introduction

Observability data are now a material line item in the cost of running cloud services. In a 2025 vendor-run survey of 1,255 practitioners, 74% said cost was a top priority when selecting observability tools, and 37% named "costs too much" as a concern [1]. Among the 294 respondents who answered an optional question on spend, observability averaged 17% of total compute infrastructure spend, with 10% as the most common answer [1]. An earlier vendor report found that 40% of respondents saw rapid data growth significantly affect their observability bill [2]. The 2022 CNCF observability micro-survey listed hidden costs among the main concerns about wider adoption [3]. These surveys are vendor-run or small, and we cite them only as evidence that cost pressure is widespread. They do not measure what is lost when telemetry is reduced.

The standard ways to cut telemetry volume are well known. Distributed traces are sampled, either at the root of a request (head-based sampling) or after the trace completes (tail-based sampling) [4,5]. Metrics are scraped less often. High-cardinality labels such as customer or tenant identifiers are dropped, aggregated, or limited, as metric-system guidance recommends [6]. Logs are sampled, sometimes uniformly and sometimes by severity. The OpenTelemetry documentation presents sampling as a primary lever for cutting observability cost while retaining visibility. It also lists, among the costs of sampling, the opportunity cost of missing critical information [4]. The Dapper report from 2010 noted that low-traffic workloads may miss important events at low sampling rates [5]. Practitioners therefore know that reductions have a detection cost. What is missing is a way to quantify that cost: which reductions delay detection, which prevent it, for which kinds of incidents, and at what saving.

The trade-off between monitoring accuracy and monitoring overhead is a long-standing concern of network and service management. Packet sampling standards define how to select a representative subset of traffic [7]. Adaptive polling schemes for software-defined networks adjust how often statistics are collected to balance accuracy against controller and switch load [8–10], and selective in-band telemetry samples which packets carry measurements [11,12]. Service-level telemetry in cloud-native systems, meaning distributed traces, labelled metrics and logs, faces the

same trade-off with two differences. Its cost is billed per byte or per time series, and the quantity operators care about is not estimation error but whether, and how soon, an alert fires.

This paper addresses that gap with a reproducible simulation. Its central question is: *for a given incident class, how do trace sampling rate, tail-sampling configuration, metric scrape interval, metric cardinality caps and log sampling rate change the probability and delay of detection, and what does each setting cost?* We hold the alert logic fixed across pipelines so that differences are attributable to the data each pipeline retains, not to differences in rule design. We make six contributions.

1. **A simulation framework.** A seeded discrete-time simulator of a heavy-tailed microservice system, five incident classes and five families of telemetry reduction, evaluated under identical, precisely defined alert rules (Sections 3 and 4). It is implemented in about 1,300 lines of Python (Section 4.5), and the code and raw results are provided as Supporting Information S1.
2. **Measurements.** Detection probability, time to detect and false-alert rate for 43 single pipeline configurations (18 trace and log, 25 metric) and 1,247 combinations of them, over 2,500 incident trials and 10 incident-free simulated days (Section 6).
3. **Analytical models validated against the simulation.** Closed-form expressions for the probability that sampled telemetry captures evidence of an incident, the detection delay due to scrape interval and alert persistence, the dilution of a tenant-specific failure by cardinality aggregation, and the failure point of a memory-capped tail sampler (Section 5).
4. **A cost model and Pareto frontier.** Volumes converted to illustrative monthly cost under three public list-price books, and the resulting cost–detectability frontier (Section 6.6).
5. **Guidance.** A mechanically derived classification of each reduction as safe, delayed or lossy for each incident class (Section 8.1).
6. **Deployment mapping.** A mapping of every simulated pipeline and alert rule to OpenTelemetry SDK, OpenTelemetry Collector and Prometheus configuration, and a step-by-step procedure for choosing reduction settings from quantities an operator already measures (Section 7).

Network-management studies of sampling and polling measure estimation accuracy, controller overhead or the success of traffic anomaly detectors (Section 2.1). To our knowledge, no peer-

reviewed paper directly measures alert detection delay as a function of trace sampling rate, scrape interval or cardinality caps in cloud systems. Our literature search was not exhaustive, and Section 2 positions the closest work. The study is synthetic. It uses no production traces, and its quantitative results depend on parameters we chose and state. We therefore emphasise mechanisms and scaling relationships, which the analytical models make portable, over the specific numbers.

2 Background and related work

We position the work against four bodies of literature: monitoring accuracy versus overhead in network and service management, distributed-trace sampling, metric and log reduction in cloud-native monitoring, and the effect of sampling on detection. Table 1 summarises how they differ from this study.

2.1 Monitoring accuracy versus overhead in network and service management

Choosing how much monitoring data to collect is a long-standing problem in network and service management. The IETF packet sampling work distinguishes content-independent sampling, such as systematic or uniform random selection, from content-dependent selection and filtering on packet properties [7]. The network telemetry framework classifies the techniques used to collect data from the management, control and forwarding planes [13]. These categories map directly onto the reductions we study. Head-based trace sampling and uniform log sampling are content-independent; tail sampling and severity-aware logging select on content.

In software-defined networks, pull-based statistics collection makes the polling interval the main lever between accuracy and overhead. OpenNetMon adapts its polling rate to flow dynamics, reducing switch and controller overhead while keeping measurements accurate [8]. PayLess adapts per-flow polling timeouts and reports accuracy close to periodic polling with about half the messages (6.6 against 13.5 per second) [9]. Tangari et al. propose a self-adaptive, decentralized monitoring framework that lowers overhead while preserving the performance of the managed services [10]. Work in this journal has optimised flow-statistics collection for cost and accuracy [14], minimised flow monitoring costs in large software-defined networks [15] and evaluated learning algorithms for dynamic real-time monitoring [16].

In-band network telemetry raises the same trade-off at packet granularity. Sel-INT samples telemetry insertion at runtime and shows that rates below 20% markedly improve switch

throughput, while monitoring accuracy remains relatively high at a sampling rate of 7.1% [11]. LINT reduces data-plane overhead by about 25% while keeping recall above 0.9 for its monitoring queries [12], and D-INTO treats telemetry orchestration as a balance between measurement accuracy and network stability [17]. For datacenter metrics, Yaseen et al. found that 89% of 1,613 metric–device pairs were sampled faster than their Nyquist rate and argued for signal-aware collection [18].

These studies measure accuracy as estimation error, as the recall of monitoring queries or as the quality of a downstream control decision. They do not evaluate alert rules on the reduced data, and they do not address the cloud-native signals (traces, labelled metrics and logs) whose cost is billed per byte or per series. Our evidence model (Section 5.1) is in the same spirit as these accuracy–overhead analyses, but its output is the probability and delay of an alert.

2.2 Trace sampling

Dapper established uniform head-based sampling for production tracing. Its first production version sampled on average one trace in 1,024, and its authors reported sampling probabilities as low as 0.01% for high-traffic services [5]. Dapper also reported that each span averaged 426 bytes and that collection latency had a median under 15 s [5]. Canopy processes more than a billion traces per day and lists trace sampling as one of its core challenges [19]. A line of work argues that uniform sampling wastes its budget on common-case traces. Sifter biases sampling toward unusual code paths without feature engineering [20]. STEAM aims to preserve observability under sampling and attributes the failure of uniform sampling to the long-tail distribution of traces [21]. TraceMesh uses locality-sensitive hashing and evolving clustering to sample streams [22], and Sieve uses attention-based sampling [23]. Hindsight retrieves trace data lazily, only after symptoms are detected. Its authors contrast head sampling, which cannot know in advance which requests will matter, with tail sampling, which has high overhead and ingestion cost [24]. Mint reduces trace storage to an average of 2.7% while still capturing all requests, and reports that a keep-or-drop ("1 or 0") sampling strategy fails to balance cost and information [25].

Two recent systems come from the network-management venues. SampleHST selects traces on the fly with a forest of half-space trees under an explicit storage budget [26]. SampleHST-X extends it to collective anomalies with approximate trees and reports 1.2 to 19 times better sampling performance than a recent trace sampler on a microservice benchmark [27].

Several studies evaluate sampling by its effect on downstream diagnosis. TraStrainer adapts sampling to system runtime state and reports an average 32.63% increase in top-1 root-cause-analysis accuracy over four baselines [28]. Trace Sampling 2.0, a preprint, uses code knowledge for span-level sampling and reports an 81.2% reduction in trace size with 98.1% faulty-span coverage [29]. UniSage, also a preprint, samples traces and logs jointly. Its second version reports that at a 2.5% sampling rate it captures 71% of critical traces and 96.25% of relevant logs [30]. Ertl derives unbiased estimators for partially sampled traces with per-span sampling rates [31]. These studies measure diagnostic accuracy or coverage after an incident is known. Our study asks the earlier question: whether, and how quickly, an alert fires at all.

2.3 Metrics, cardinality and log reduction

Time-series monitoring systems are designed around aggregate analysis of recent data. Gorilla's designers observed that recent data points are worth much more than older ones for detecting and diagnosing an ongoing problem [32]. Monarch ingests terabytes of time-series data per second [33]. Prometheus scrapes targets at a configurable interval (default 1 minute) and evaluates rules at a configurable interval (also 1 minute by default). It provides cardinality guards such as *sample_limit*, whose violation fails the entire scrape rather than truncating it [34]. Recording-rule and alerting-rule groups have a *limit* beyond which all series or alerts produced by the rule are discarded [35]. Alerting rules accept a *for* clause that keeps an alert pending until its condition has held for the stated duration [36]. Prometheus guidance warns against labels with unbounded values, such as user identifiers, because each label combination is a new series [6]. In the IoT setting, AdaM adapts metric sampling rates and reports a 74% data reduction while keeping accuracy above 89% [37].

For cloud-native metrics, AdapPF adapts the scrape interval of Prometheus federation to cluster load and reports scheduling accuracy comparable to a 5 s interval with up to 36% less cross-cluster traffic [38]. AdaMC adapts scrape frequency for microservices in edge-cloud settings and reports resource savings over fixed-interval Prometheus scraping [39]. MicroRCA localises the root causes of microservice performance issues from continuously collected application and system metrics [40]. Like most diagnosis methods, it assumes the telemetry it needs has been collected.

For logs, LogReducer identifies the small number of log templates that dominate overhead and reports a 39.08% reduction in daily storage in a production system [41]. Logzip compresses logs

by extracting hidden structure [42]. LogCleaner reports that removing over 70% of log events can improve anomaly-detection models [43]. Targeted reduction is therefore not inherently harmful. Karumuri et al. argue for purpose-built observability data management systems because of the volume and cost involved [44].

2.4 Sampling and detectability

The closest peer-reviewed quantifications of sampling versus detectability come from network measurement. Brauckhoff et al. found that packet sampling preserves anomaly size in byte and packet counts but grossly biases flow counts [45]. Ishibashi et al. studied the effect of sampling rate and monitoring granularity on anomaly detectability [46]. On alerting, the SRE Workbook evaluates alert designs by precision, recall, detection time and reset time. It shows that a duration condition delays detection equally for large and small outages, and recommends against durations in SLO alerting [47].

Mai et al. sampled backbone traces with four methods and found that all of them degraded the detection of volume anomalies and port scans. Methods biased toward heavy hitters missed the small flows of which many attacks consist [48]. The low-traffic endpoints and small tenants in our study play the role of those small flows.

2.5 Positioning

Table 1 Positioning of this study against related lines of work.

Line of work	Telemetry	What is reduced or adapted	Measure of success	Alert rules held fixed	Monetary cost
Adaptive polling in SDN [8–10,14,15]	Flow counters	Polling frequency, switches polled	Estimation error, messages, controller load	No	No
Selective in-band telemetry [11,12,17]	Per-packet telemetry	Packets carrying telemetry	Throughput, query recall, stability	No	No
Packet sampling and anomaly detection [45,46,48]	Packets, flow records	Sampling method and rate	Detector success and false positives	Yes	No
Metric collection rate [18,37–39]	Time-series metrics	Scrape or sampling interval	Reconstruction or scheduling accuracy, traffic	No	No
Trace sampling [20,21,24–28]	Distributed traces	Which traces to keep	Anomalous-trace coverage, diagnosis accuracy, storage	No	Storage only
This study	Traces, labelled metrics, logs	Head and tail sampling, scrape interval, cardinality cap, log sampling	Detection probability and delay, false alerts	Yes	List-price cost

The table summarises each line of work as cited; individual papers may go beyond the summary.

Table 1 shows where the gap lies. Packet-sampling studies come closest: they hold detectors fixed and measure detection under sampling. They concern network traffic, however, not the service-level signals that operators of cloud services now reduce, and they do not consider cost. Trace-sampling studies optimise which traces to keep and judge success after an incident is known. Practitioner documentation acknowledges that reductions have an opportunity cost [4]. We found no study that holds alert rules fixed and measures how each standard reduction setting for traces, metrics and logs changes detection probability and delay across incident classes, together with what it saves. That is the contribution of this paper. The author's concurrent work, which is unpublished and currently under review, studies a related averaging effect in fleet-wide burn-rate alerting across edge sites. The present paper is distinct: it studies how telemetry reduction removes evidence before any alert rule runs, and it covers sampling, scrape intervals, cardinality and logs.

3 System and incident model

We simulate in discrete time with 5-second bins. In each bin we draw request and error counts for every endpoint from distributions described below, and the telemetry pipelines (Section 4) observe these counts. The simulator is written in Python with NumPy [49]. Statistical post-processing uses SciPy [50]. Every random quantity is drawn from a seeded generator. Table 2 summarises the model.

3.1 Call graph and traffic

The system has 30 services in four layers (4 gateway, 8 mid-tier, 11 backend and 7 data services), with 1 to 4 endpoints per service and 59 endpoints in total. Endpoints on gateway services are the roots of traces. There are nine root operations, and their arrival rates are drawn from a log-normal distribution ($\sigma = 1.2$) and scaled to 1,000 traces per second in total. Each non-leaf endpoint calls 1 to 5 endpoints in deeper layers. Each call edge is unconditional with probability 0.6. Otherwise it is taken with a probability drawn log-uniformly from 1% to 50%, which models cache-miss or fallback paths. Endpoints left without a caller receive one rare edge, with call probability 1–10%. Propagating root rates through the graph gives 9,137 spans per second and a mean of 9.1 spans per trace. Endpoint rates are heavy-tailed: from 0.024 to 914 requests per second, with quartiles of 5.2, 58 and 327 per second. A common-mode load factor follows a log-AR(1) process with a 10% standard deviation and a 10-minute correlation time. Per-bin request counts are Poisson.

3.2 Errors, latency and tenants

Each endpoint has a baseline error probability drawn log-uniformly from 0.02% to 0.28%. The span-weighted mean is 0.113%. Each endpoint's self-time is log-normal, with a median between 2 and 80 ms and a shape parameter between 0.4 and 0.6. A span's duration is its self-time plus the durations of the calls it makes, which are treated as sequential. A Monte Carlo pass of 100,000 traces per root operation gives each endpoint's span-duration distribution. We set each endpoint's latency SLO threshold T_j to its baseline 99.9th percentile (median 242 ms), so that 0.1% of spans are slow at baseline. A rule that fires when more than 1% of spans exceed T_j is therefore exactly a rule that fires when the p99 exceeds T_j.

Every request belongs to one of 2,000 tenants with Zipf-distributed shares (exponent 1.1). The largest tenant carries 16.9% of traffic, the tenth 1.34% and the two-hundredth 0.050%. Tenant membership is independent of endpoint.

3.3 Incident classes

Each incident trial runs 20 minutes of warm-up, then the incident, then 10 minutes of post-incident observation. An incident counts as detected if an alert on a relevant series begins to fire between incident onset and 10 minutes after the incident ends. Relevant series are the affected endpoint, any endpoint for the global class, and, for the tenant class, the tenant's series, the aggregate series containing it, or any endpoint. Time to detect (TTD) is measured from onset. We inject five classes with 500 trials each.

- **I1, global error spike.** Every endpoint's error probability rises by 10 percentage points for 15 minutes. This models a shared infrastructure failure.
- **I2, low-traffic endpoint error.** An endpoint drawn uniformly from the lowest-traffic quartile fails 20% of its requests for 30 minutes. Target rates range from 0.024 to 4.5 requests per second, with a median of 0.53.
- **I3, latency regression on one dependency.** A non-root endpoint from the middle two traffic quartiles (5.8 to 303 requests per second) sends 5% of its requests down a slow path for 30 minutes. The slow path adds self-time equal to the endpoint's SLO threshold, raising the fraction of slow spans from 0.1% to 4.9–5.2% and pushing the p99 above threshold.

- **I4, gradual error creep.** An endpoint from the middle two quartiles has an error probability that rises linearly from baseline to 10% over 120 minutes. Because the rise is gradual, we also report delay relative to the moment the true error ratio crosses the 2% alert threshold (the *detectable onset*). That moment is a mean of 1,382 s after the incident starts.
- **I5, single-tenant failure.** A tenant with rank drawn uniformly from 11 to 200 fails 30% of its requests on every endpoint for 30 minutes. Its share of traffic ranges from 0.050% to 1.21%.

Each trial uses a fresh random target and fresh traffic. All pipelines in a trial observe the same ground truth, so pipelines can be combined exactly by taking the earliest alert across members.

3.4 Incident-free runs and benign transients

False alerts are measured on 40 incident-free runs of 6 hours each, a total of 10 simulated days. To give the rules something realistic to reject, these runs include benign transients. Each happens six times per day on average. Error bursts occur at a random endpoint, latency bursts at a random endpoint, and error bursts at a random tenant among the 500 largest. Each burst lasts 10–60 s and affects 5–30% of requests. Transients are not injected in incident trials, so that detections there can be attributed to the injected incident.

Table 2 Simulation model, telemetry pipelines and alert rules.

Element	Setting
System	30 services, 59 endpoints, 9 root operations; 1,000 traces/s; 9,137 spans/s (9.1 spans/trace); endpoint rates 0.024–914 req/s (median 58); log-AR(1) common load factor, sd 10%, 10-min correlation
Errors, latency	Baseline error 0.02–0.28% per endpoint; log-normal self-times; SLO threshold T_j = baseline p99.9 of span duration
Tenants	2,000, Zipf exponent 1.1 (largest 16.9%, rank 10: 1.34%, rank 200: 0.050%)
(a) Head sampling	Trace kept with probability $s \in \{100, 50, 20, 10, 5, 2, 1, 0.5, 0.2, 0.1\}$%; 15 s export delay
(b) Tail sampling	Keep trace if it has an error span, or root duration > operation's baseline p99, or 1% probabilistic; decision wait 30 s; FIFO buffer of C = 50,000 traces (headroom swept separately); 15 s export delay
(c) Metrics	Per endpoint: ok/error counters and a 12-bucket latency histogram with a boundary at T_j (16 series); scrape interval $\Delta \in \{10, 30, 60, 120, 300\}$ s; rate window $w = \max(5 \text{ min}, 2\Delta)$; increases extrapolated to the window
(d) Cardinality cap	Tenant label on request/error counters (2 series per active endpoint–tenant pair); top-N tenants by volume kept, others aggregated to "other"; $N \in \{0$ (dropped), 10, 100, 500, 2,000 (uncapped)$\}$

Element	Setting
(e) Log sampling	One access line (300 B) per request, one error line (1,200 B) per failure; uniform rate r ∈ {100, 10, 1, 0.1}% or severity-aware (all lines of failed requests kept, others at r); 15 s export delay
Alert rules (all pipelines)	Error rule: estimated error ratio over w > 2% and ≥ 5 retained error events in w. Latency rule: estimated fraction of spans above T_j > 1% and ≥ 5 retained slow events. Tenant rule: error rule per tenant series. All: evaluated every 15 s, for: 2m; w = 5 min for event pipelines
Experiments	500 trials per incident class; 40 × 6 h incident-free runs; tail-buffer sweep (166 trials per point); tenant-rank sweep (83 trials per rank)

Payload sizes for logs are illustrative assumptions; span size (426 B) is the Dapper average [5].

4 Telemetry pipelines and alert rules

Figure 1 shows the structure of the study. Each pipeline transforms the ground-truth counts into the data an alerting system would see. Every pipeline is then evaluated with the same three rules.

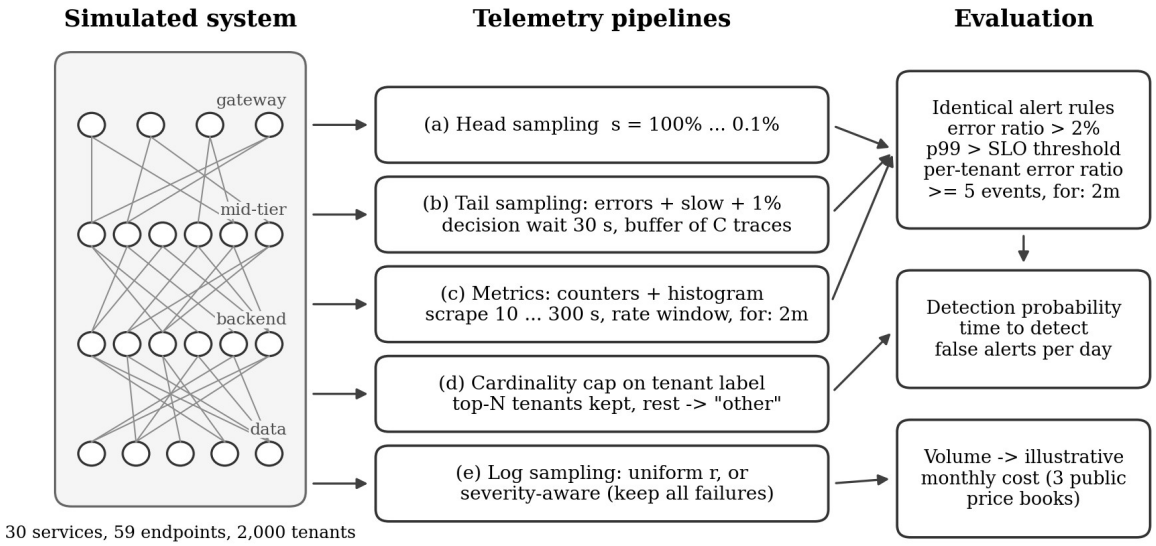


Fig. 1 Study design. A seeded microservice simulation produces ground-truth request, error and latency events. Five families of telemetry reduction retain different subsets. Identical alert rules are applied to each pipeline's output, and the results are summarised as detection probability, time to detect, false alerts and illustrative cost

4.1 Event pipelines: traces and logs

For head-based sampling at rate *s*, each trace is kept independently with probability *s*. For per-endpoint rules this thins each endpoint's spans binomially. Uniform log sampling at rate *r* is statistically identical for per-endpoint rules, because each request contributes one access line. We simulate it independently and confirm the equivalence (Table 3). Severity-aware log sampling keeps every line of a failed request and samples the remaining lines at rate *r*.

The tail sampler follows the policy types of the OpenTelemetry Collector's tail-sampling processor [51]. It keeps a trace if any span has error status, if the root duration exceeds the operation's baseline p99 (a per-operation latency policy), or with 1% probability. The decision is made 30 s after the trace's first span, which is the processor's default *decision_wait* [51]. Pending traces occupy a buffer whose default capacity (*num_traces*) is 50,000 [51]. We model the buffer as first-in, first-out. A trace is lost if 50,000 newer traces arrive before its decision. At our load, the default buffer leaves about 1.67 times the mean occupancy of 30,000 traces, and we sweep this headroom in Section 6.4.

A tail sampler's latency policy acts on trace duration, but the I3 regression is a property of one span. Whether a slow span is retained therefore depends on whether it makes its whole trace slow. The Monte Carlo pass estimates, for each endpoint, the fraction ρ_j of slow spans whose trace exceeds the root operation's threshold. The median ρ_j is 0.99, but it falls to 0.017 for some deep endpoints whose latency is small relative to the trace. A slow span with no error is retained with probability $\rho_j + (1 - \rho_j) \cdot 0.01$. We do not model spans that are kept only because another span in the same trace failed. This is conservative for the tail sampler.

4.2 Estimating ratios from sampled events

Rules compare ratios, so sampled counts must be reweighted. We use Horvitz–Thompson weights [52], meaning each retained event is weighted by the inverse of its inclusion probability. Under head and uniform sampling, every event has weight $1/s$, so the ratio is simply the ratio of retained counts. Under tail sampling, spans kept by the error or latency policy have weight 1 and spans kept only by the probabilistic policy have weight 100. Under severity-aware logging, failure lines have weight 1 and other lines have weight $1/r$. This mirrors the adjusted-count approach used to estimate rates from sampled spans [31]. Its weakness matters later: when few non-failure events are kept, the denominator is estimated from very few samples and the ratio becomes noisy.

4.3 Metric pipelines and cardinality caps

Metric pipelines see complete counts but only at scrape times. Every Δ seconds, at a random phase, the scraper samples cumulative counters. At each evaluation, the rule takes the first and last samples inside the window $w = \max(5 \text{ min}, 2\Delta)$, computes their increases, and extrapolates them to the full window as the PromQL *increase()* function does. The error rule needs at least two

samples in the window. We assume the histogram has a bucket boundary at T_j, which is common practice for SLO latency histograms, so the slow fraction is observed exactly.

The tenant label is placed on request and error counters, giving two series per endpoint–tenant pair. A pair counts as active if it was seen in the last 24 hours. A cap of N keeps the label value for the N largest tenants and maps all others to the value "other". This is the most favourable form of capping. Prometheus's own *sample_limit* instead fails the whole scrape when exceeded [34], which would blind every rule on that target. We discuss that case in Section 8.2.

4.4 Alert rules

All pipelines use the same three rules (Table 2). The error rule fires when the estimated error ratio over the window exceeds 2% and at least $k = 5$ error events were retained in the window. The latency rule fires when the estimated fraction of spans slower than T_j exceeds 1% (equivalently, p99 > T_j), with the same evidence guard on slow events. The tenant rule applies the error rule to tenant series. Rules are evaluated every 15 s and include *for: 2m*: the condition must hold on nine consecutive evaluations before the alert fires [36]. Event pipelines use a 5-minute window over events that have reached the backend, which happens 15 s after the event for traces and logs and 45 s after for tail-sampled traces. The evidence guard k was tuned once, on the full-fidelity pipeline, to keep false alerts on low-traffic endpoints and small tenants at a few per day. It was then held fixed, as an operator who tuned alerts on unreduced telemetry would do.

4.5 Implementation

The simulator is about 1,300 lines of Python in five modules, following the data flow of Figure 1. *model.py* builds the seeded topology, the rate, latency and error processes and the tenant population, and runs the Monte Carlo pass that gives span-duration distributions, SLO thresholds and the latency-policy retention probabilities ρ_j. *run_sim.py* generates the ground truth of each trial as arrays of per-bin request, error and slow-span counts for every endpoint, and for tenant series where needed, and drives the experiments. *sim.py* implements every pipeline as a thinning or aggregation of those arrays and evaluates the alert rules. *analyze.py* computes every reported number, the checks of the analytical models, volumes, costs and the combination frontier, and *figures.py* draws the figures.

Three design choices keep the study tractable and exact. First, all pipelines in a trial transform the same ground-truth arrays, so the 43 configurations of a trial share one draw of traffic and incident,

and any combination of pipelines can be scored afterwards by taking the earliest alert. Second, rule evaluation uses cumulative sums over the 5 s bins, so every windowed count is a constant-time difference, and the *for* clause is a run-length test on the sequence of evaluations. The metric pipeline reproduces the scrape phase, the first and last samples in the window and the extrapolation of PromQL *increase()*. Third, the tail sampler is the only stateful component. For each bin it computes the fraction of traces that survive until their decision, given the number of traces arriving during the decision wait and the buffer capacity, which implements the FIFO eviction of Section 5.4 without simulating individual traces.

Seeds are derived from two master seeds, one for the topology and one for the experiments, and each trial has its own named generator, so that any stage can be re-run on its own and reproduce the same numbers. On a four-core virtual machine the experiment stages took about 15 minutes in total: about 2.5 minutes for the 2,500 incident trials and about 11 minutes for the 40 incident-free runs. The simulator, the analysis and figure scripts, the raw per-trial outputs and the results file (results.json) that supports every reported result are provided as Supporting Information S1 (paperF_code.zip). During the preparation of this work the author used Claude (Anthropic) to assist with drafting and editing text and writing analysis code. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the article.

5 Analytical models

Four simple models explain most of the behaviour we observe. Section 6 validates each against the simulation.

5.1 Capture and evidence under sampling

Consider an endpoint with request rate λ, error probability e during an incident of length W, and a pipeline that retains each failed request independently with probability s. Retained errors then form a Poisson process with rate $\mu = \lambda e s$. The probability that at least one error trace is retained is

$$P_capture = 1 - exp(-\lambda e s W) \qquad (1)$$

and the probability that at least k are retained is

$$P(N \geq k) = 1 - \Sigma_\{i<k\} (\mu W)^i e^{-\mu W} / i! \qquad (2)$$

An alert needs more than retained evidence. It needs k retained events inside one rule window w, sustained for the *for* duration. The governing quantity is therefore the dimensionless evidence

$$x = \lambda\, e\, s\, w / k \qquad (3)$$

This is the expected number of retained incident errors per window, divided by the guard. Detection depends on λ, e and s only through x. The minimum safe sampling rate for an endpoint is about $s^* = k / (\lambda\, e\, w)$. For example, an endpoint serving 0.5 requests per second that fails 20% of requests needs $s^* \approx 5 / (0.5 \times 0.2 \times 300) \approx 17\%$ under our rules. A head sampler set to 1% cannot detect that failure. The same reasoning applies to uniform log sampling and, for latency, to the slow fraction in place of *e*.

5.2 Delay due to scrape interval and persistence

Suppose the error ratio steps from e_0 to e_1 at time 0, with threshold θ between them. A windowed ratio exceeds θ once a fraction $f = (\theta - e_0)/(e_1 - e_0)$ of its coverage lies after the onset. For metrics, coverage is the span between the first and last scrape samples in the window, which is $w - \Delta$ on average. The condition becomes true at the first scrape at least $f(w - \Delta)$ after onset. That scrape comes on average Δ/2 later because the scrape phase is random. The alert then fires one evaluation lag δ and the *for* duration D later:

$$E[TTD_{metrics}] \approx f\,(w - \Delta) + \Delta/2 + \delta + D \qquad (4)$$

For event pipelines there is no scrape term, but there is a pipeline delay d (plus the 30 s decision wait for tail sampling): $E[TTD] \approx d + f\,w + \delta + D$. For a linear creep that crosses θ at time t_c, the windowed ratio equals the true ratio at the window's midpoint, so

$$E[TTD_{creep}] \approx t_c + (w - \Delta)/2 + \Delta/2 + \delta + D \qquad (5)$$

Equations (4) and (5) show that scrape interval adds about Δ/2 directly. It also changes coverage, and the coverage effect partly cancels the Δ/2 term while Δ stays below the rule window. The *for* clause adds its full duration to every detection, as the SRE Workbook notes for duration conditions [47].

5.3 Dilution by cardinality aggregation

Let a tenant with traffic share σ fail with error probability e_t while all other traffic fails at e_0. If the tenant's requests are aggregated into a series that also carries a share σ_{rest} of healthy traffic, the series' error ratio is $(\sigma\, e_t + \sigma_{rest}\, e_0)/(\sigma + \sigma_{rest})$. It exceeds θ only if

$$\sigma / (\sigma + \sigma_{rest}) > \varphi = (\theta - e_0)/(e_t - e_0) \qquad (6)$$

With θ = 2%, *e_t* = 30% and *e_0* = 0.113%, φ = 6.3%. A tenant aggregated into the "other" bucket is detectable only if it carries more than 6.3% of that bucket's traffic. If the tenant label is dropped entirely, the tenant must carry about 5.8% of each endpoint's traffic, a figure computed using the endpoint with the highest baseline error rate. In our Zipf population that means only the two largest tenants. The same averaging structure underlies the fleet-wide dilution bound in the author's concurrent, unpublished work mentioned in Section 2.5. Here, however, it is caused by a telemetry reduction, not by the choice of alerting scope.

5.4 Memory-capped tail sampling

A FIFO buffer of capacity *C* with decision wait *W_d* loses a trace whenever *C* newer traces arrive before its decision. At a steady arrival rate Λ, every trace is lost once Λ·W_d > C, and none is lost below that. The relevant quantity is the headroom $h = C/(\bar{\Lambda} W_d)$ relative to the mean load $\bar{\Lambda}$. When load fluctuates, losses begin as soon as peaks exceed C/W_d. The tail sampler therefore fails abruptly, not gradually, as headroom falls through about 1 plus the peak-to-mean load ratio. This follows from the statefulness the OpenTelemetry documentation warns about: tail sampling must hold every span of a trace in one place until the decision [4,51].

6 Results

6.1 Detection by incident class

Table 3 reports detection probability and median TTD for each pipeline and class. Each cell is also classified against the 100% trace pipeline using a fixed rule. A cell is *lossy* (L) if detection probability falls by more than 0.05, *delayed* (D) if it does not but median TTD rises by more than 60 s, and *safe* (S) otherwise. Figure 2 plots the main trends.

Table 3 Detection probability, median time to detect (s), and classification against 100% traces (S safe, D delayed, L lossy). 500 trials per class. False alerts per day from 10 incident-free days.

Pipeline	I1 global	I2 low-traffic	I3 p99 dep.	I4 creep	I5 tenant	False/day
Traces 100%	1.00 · 175	0.90 · 190	1.00 · 195	1.00 · 1675	1.00 · 160	10.0
Head 10%	1.00 · 175 S	0.49 · 305 L	1.00 · 200 S	1.00 · 1700 S	1.00 · 165 S	7.7
Head 1%	1.00 · 170 S	0.05 · 858 L	0.72 · 235 L	0.79 · 1860 L	0.99 · 305 D	4.3
Head 0.1%	1.00 · 175 S	0.00 · – L	0.19 · 878 L	0.38 · 5095 L	0.16 · 605 L	0.9
Tail (errors+slow+1%)	1.00 · 200 S	0.90 · 225 S	1.00 · 230 S	1.00 · 1705 S	1.00 · 190 S	16.1
Logs uniform 10%	1.00 · 170 S	0.50 · 320 L	1.00 · 200 S	1.00 · 1685 S	1.00 · 165 S	7.2
Logs uniform 1%	1.00 · 170 S	0.05 · 1208 L	0.72 · 240 L	0.80 · 1870 L	0.99 · 305 D	5.0

Pipeline	I1 global	I2 low-traffic	I3 p99 dep.	I4 creep	I5 tenant	False/day
Logs severity 1%	1.00 · 170 S	0.90 · 195 S	0.72 · 245 L	1.00 · 1680 S	1.00 · 160 S	9.7
Logs severity 0.1%	1.00 · 150 S	0.90 · 215 S	0.17 · 768 L	1.00 · 1600 S	1.00 · 160 S	347.3
Metrics 10 s	1.00 · 165 S	0.89 · 175 S	1.00 · 185 S	1.00 · 1660 S	1.00 · 150 S	10.5
Metrics 60 s	1.00 · 180 S	0.91 · 205 S	1.00 · 200 S	1.00 · 1655 S	1.00 · 170 S	13.2
Metrics 120 s	1.00 · 205 S	0.92 · 230 S	1.00 · 220 S	1.00 · 1655 S	1.00 · 200 S	16.0
Metrics 300 s	1.00 · 310 D	0.96 · 365 D	1.00 · 325 D	1.00 · 1800 D	1.00 · 290 D	77.7
Metrics 30 s, cap 100	1.00 · 170 S	0.89 · 190 S	1.00 · 190 S	1.00 · 1655 S	0.44 · 155 L	7.6
Metrics 30 s, label dropped	1.00 · 170 S	0.89 · 190 S	1.00 · 190 S	1.00 · 1655 S	0.00 · – L	6.7

Metric rows without a cap carry the uncapped tenant label. I4 times are from incident start; the detectable onset is a mean 1,382 s later (Section 3.3).

The global spike (I1) survives every reduction. All pipelines detected all 500 global incidents, with median TTD between 150 and 310 s. Even 0.1% head sampling retains enough error spans from high-traffic endpoints to pass the evidence guard quickly. For I1, the dominant source of delay is the rule itself: the 5-minute window, which needs a fraction $f \approx 0.2$ of its coverage to show the elevated ratio, and the 2-minute *for* clause. Telemetry reduction contributes little.

Low-traffic endpoint failures (I2) are where head sampling fails. At full fidelity the error rule detected 89.8% of I2 incidents (95% Wilson interval 86.8–92.2%). The misses are concentrated on the slowest endpoints. Among the 243 trials whose target served fewer than 0.5 requests per second, detection was 79%, against 100% for the rest. Head sampling at 10% reduced detection to 48.8% (44.4–53.2%), 1% to 4.8%, and 0.1% to zero. Uniform log sampling behaved identically, as expected (50.2% at 10%, 4.8% at 1%). In contrast, both reductions that keep every failure preserved I2 detection completely. Tail sampling detected 89.8%, with median TTD rising from 190 to 225 s. Severity-aware logs detected 89.8% at 1% and 89.6% at 0.1%. Metrics detected 89.2–95.6%. Longer scrape intervals slightly increased I2 detection (from 89.4% at 10 s to 95.6% at 300 s) because the rule window grows to 10 minutes at $\Delta = 300$ s and accumulates more evidence. The cost was a median TTD of 365 s instead of 175 s.

The dependency latency regression (I3) separates the tail sampler from severity-aware logs. Slow requests are not failures, so severity-aware log sampling treats them like any other success. At 1% it detected 71.8% of I3 incidents, the same as 1% head sampling (72.4%) and 1% uniform logs (72.2%). The tail sampler detected all I3 incidents. Its latency policy retains most slow spans because, for most endpoints, a slow span makes its trace slow (median ρ_j 0.99). For the minority

of deep, fast endpoints where it does not, the 1% probabilistic component still supplies enough evidence at the I3 traffic levels. Metrics at any scrape interval detected all I3 incidents, because the histogram bucket at T_j counts every slow request.

The gradual creep (I4) is limited by the rule, not the telemetry, until sampling is severe. Median TTD at full fidelity was 1,675 s, against a mean detectable onset of 1,382 s. The lag of about 5 minutes is what Equation (5) predicts from the half-window and the *for* clause. Head sampling at 1% reduced detection to 79.4% and delayed it to 1,860 s. At 0.1% detection fell to 38.2%. Metrics detected every creep at every scrape interval.

Tenant failures (I5) depend on where the tenant identifier lives. When the tenant is a span or log attribute, sampling behaves as for any other low-rate series. Tail sampling and severity-aware logs detected every I5 incident. Head sampling at 1% detected 99.2%, but 145 s later than full fidelity, and at 0.1% it detected 16%. For metrics, the deciding factor was the cardinality cap, not the scrape interval (Section 6.5).

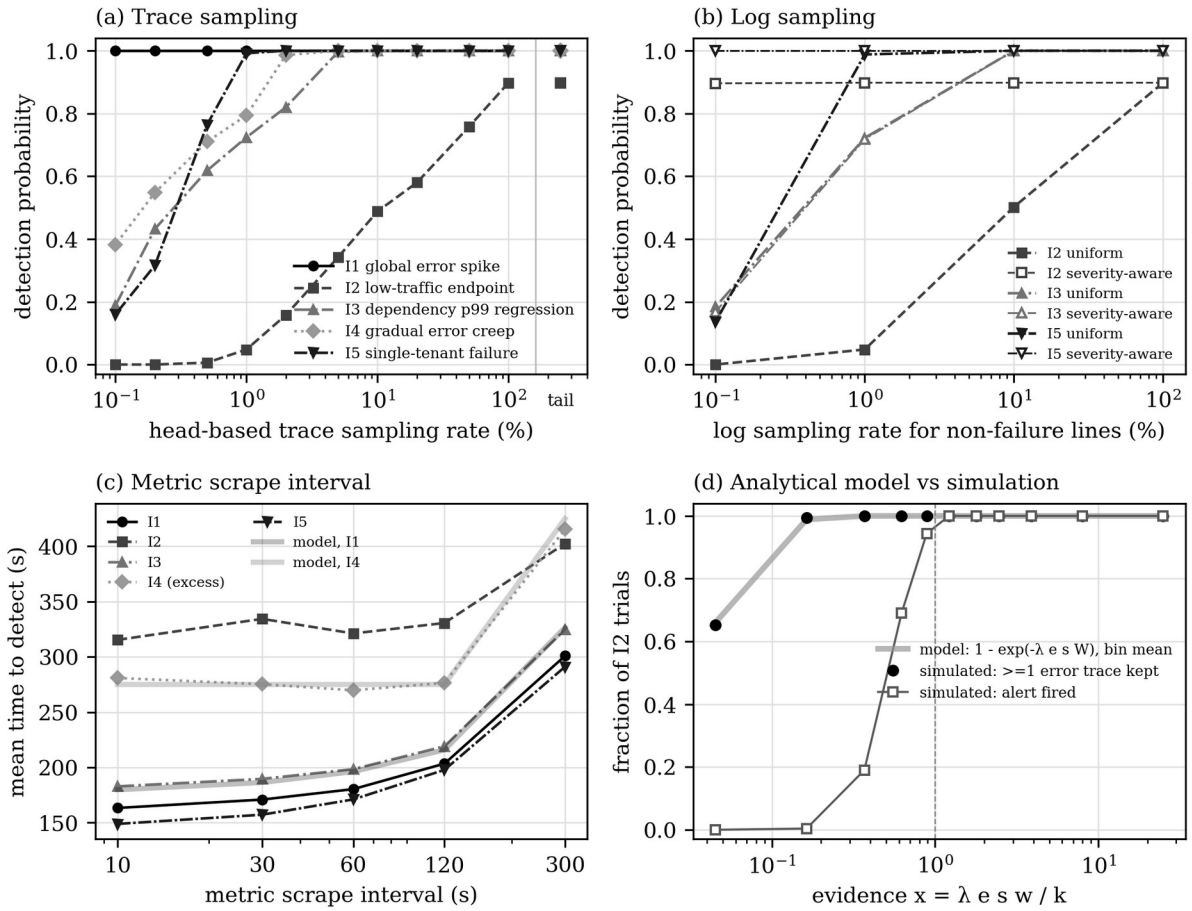


Fig. 2 (a) Detection probability against head-based trace sampling rate for the five incident classes; markers at right show tail sampling. (b) Uniform (filled) versus severity-aware (open) log sampling for I2, I3 and I5. (c) Mean time to detect against metric scrape interval, with the predictions of Equations (4) and (5) for I1 and I4 (I4 shown as excess over the detectable onset). (d) Validation of Equation (1) and of the evidence scaling (3) on I2 trials pooled over ten sampling rates

6.2 Validation of the analytical models

Capture and evidence. For each I2 trial and each of the ten head-sampling rates, we compared the predicted probability from Equation (1) with whether the simulation actually retained at least one error trace from the target during the incident. Averaged by sampling rate, prediction and observation differed by at most 0.021. Pooled into seven bins of predicted probability, the mean absolute error was 0.011 (Figure 2d). Equation (2) for at least five retained errors differed from observation by at most 0.016 at any sampling rate. Both formulas are effectively exact here, because retained errors in the simulation are a thinned Poisson process. The value of the validation is that it confirms the simulator implements the sampling it claims to.

Evidence scaling. Detection probability depended on the trial only through the dimensionless evidence x of Equation (3). Pooling all 5,000 trial–rate pairs, detection was 0.4% for x between 0.1

and 0.25. It was 19% near $x = 0.37$, 69% near 0.62, 94% near 0.89, and 100% for every bin with $x \geq 1$. Within each x bin, trials at sampling rates of 10% or more and trials below 10% gave similar detection rates where both had at least 20 trials (for example 18% and 20% near $x = 0.37$, and 94% and 96% near $x = 0.89$). The one exception was the 0.5–0.75 bin (76% and 47%), where the lower-rate group had only 32 trials. The practical rule is therefore $x \geq 1$, that is, $s \geq k/(\lambda\, e\, w)$. Capture of a single error trace is a much weaker condition. At $x = 0.16$, 99% of trials retained an error trace but only 0.4% raised an alert. A sampled trace that exists in storage is not the same as an incident that pages someone.

Scrape-interval delay. Equation (4) predicts that the mean I1 TTD rises by 147 s as Δ goes from 10 to 300 s. The simulation shows a rise of 138 s. The absolute predictions are 13–26 s high at every Δ. This is expected: I1 is detected by whichever of 59 endpoints crosses first, and statistical noise makes the minimum over many endpoints earlier than the single-endpoint formula assumes. Equation (5) predicts the I4 mean TTD to within 10 s at every scrape interval (for example 1,657 s predicted and 1,663 s observed at Δ = 10 s; 1,807 s and 1,797 s at Δ = 300 s). Scrape intervals up to 120 s raised mean I1 TTD by only 40 s over 10 s scraping. The coverage effect in Equation (4) offsets most of the Δ/2 term while Δ is well below the rule window. Only Δ = 300 s, which forces the 10-minute window, added more than two minutes.

Cardinality dilution. Equation (6) predicts which tenants are detectable under each cap. Simulated detection agreed in every case where the tenant had its own series or was deep in the "other" bucket (Figure 3b). The disagreement is at the boundary. With the label dropped, the bound predicts that only the two largest tenants (shares 16.9% and 7.9%) are detectable, but the simulation also detected all failures of the third-largest tenant (5.1%) and 49% of those of the fifth (2.9%). At the lowest-traffic endpoints, binomial noise occasionally pushes a slightly diluted ratio over 2%. That produces an alert on an endpoint, with no tenant attribution, earlier than the deterministic bound allows. The bound is therefore conservative by a factor of about two in tenant share.

6.3 False alerts

Over 10 incident-free days, the 100% trace pipeline produced 10.0 false alerts per day: 2.4 from endpoint error rules, 3.9 from latency rules and 3.7 from tenant rules, almost all triggered by the injected benign transients. Reduction generally removed false alerts along with true ones. Head sampling at 1% produced 4.3 per day and at 0.1% produced 0.9. Two reductions increased false

alerts. The tail sampler produced 16.1 per day, with latency alerts rising from 3.9 to 9.1. Its slow-span numerator is kept almost completely, while its denominator is estimated from the 1% probabilistic sample, so the ratio swings when that sample is briefly small. Severity-aware logs at 0.1% produced 347 per day, 322 of them from tenant rules. With non-failure lines kept at 0.1%, a small tenant's denominator is often estimated from zero or one line, and a handful of ordinary errors then looks like a 100% error ratio. Metrics at 300 s produced 77.7 per day with the uncapped tenant label, and 7.4 without it. Extrapolating a 5-minute increase to a 10-minute window doubles the guard count and weakens it for small tenants. We treat any pipeline with more than twice the full-fidelity false-alert rate (more than 20 per day) as inadmissible in the Pareto analysis. That excludes two configurations: severity-aware logs at 0.1% and 300 s metrics with an uncapped tenant label.

6.4 Tail-sampler memory headroom

We swept the tail-sampling buffer from 0.8 to 2.0 times mean occupancy (166 trials per point for I2 and I3). With headroom of 1.3 or more, at least 99.8% of traces survived to a decision, and detection matched the uncapped sampler (I2 94%, I3 100% in this sample). At a headroom of 1.2, 97.5% of traces survived and I2 detection was 93%. At 1.0, which means a buffer sized exactly to mean load, only 54% of traces survived, because every load peak above the mean evicts traces. I2 detection fell to 84% and its median TTD rose to 358 s. At 0.9, 16.8% of traces survived and I2 detection was 54%. At 0.8, 1.4% survived and I2 and I3 detection fell to 11% and 18%. As Section 5.4 predicts, the sampler fails abruptly between a headroom of about 1.2, roughly two standard deviations of our load process above its mean, and 0.8. The default buffer of 50,000 traces gave a headroom of 1.67 at our load. At three times the load, the same default would sit near 0.56 and would retain almost nothing.

6.5 Cardinality caps

Figure 3a shows the I5 trials, in which tenant ranks were drawn uniformly from 11 to 200. With the uncapped tenant label (224,407 active series) or a cap of 500 (59,256 series), metrics at 30 s detected every tenant failure, with a median TTD of 155 s. A cap of 100 (12,857 series) reduced detection to 44.0%, close to the 47% of trials whose tenant ranked within the top 100. Tenants beyond the cap were diluted in the "other" bucket, which carries 28% of all traffic at that cap, and none of them was detected. Caps of 10 or dropping the label (944 series) detected none. Figure 3b

generalises this. Detection is a step at the cap, apart from the noise-driven tail for the largest tenants discussed in Section 6.2. In contrast, tail-sampled traces carrying the tenant as an attribute detected failures of tenants at every rank, including rank 2,000 (share 0.004%). One-percent head-sampled traces detected 93% at rank 200, 8% at rank 500, and none beyond.

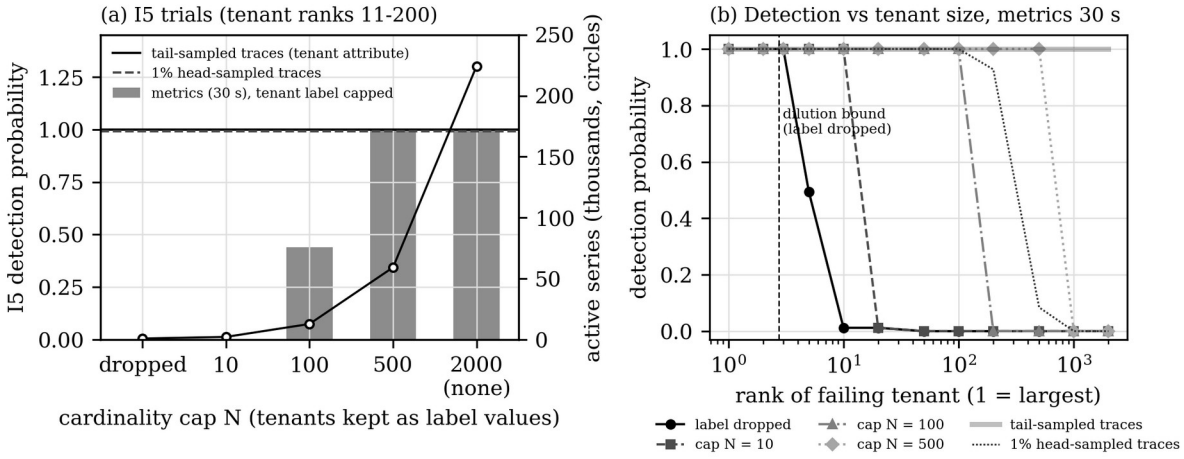


Fig. 3 Cardinality dilution. (a) I5 detection probability (bars) and active metric series (circles) as the cap on the tenant label varies, with trace-based detection for comparison. (b) Detection probability against the rank of the failing tenant for each cap (83 trials per rank). The dashed vertical line is the dilution bound of Equation (6) when the label is dropped

6.6 Volume, cost and the Pareto frontier

Table 4 converts each pipeline's steady-state volume into an illustrative monthly cost under three public list-price books, all accessed on 2026-10-05 [53–55]. For Datadog we use $0.10 per GB for span ingestion, $0.10 per GB for log ingestion, and $5 per 100 custom metrics, treating each time series as one custom metric. For Grafana Cloud Pro we use $6.50 per 1,000 active series, and $0.55 per GB for traces and for logs. The $0.55 is our sum of the listed process, write and retain components ($0.05, $0.40 and $0.10). For AWS CloudWatch we use, from the worked examples on the US East pricing page, $0.35 per GB for span ingestion (Transaction Search, first tier), $0.50 per GB for standard log ingestion, and custom metrics at $0.30 per metric for the first 10,000, $0.10 up to 250,000, and $0.05 beyond.

These are list or "starts at" prices. They exclude discounts, commitments, free allotments, platform and host fees, span indexing and retention tiers, and the compute cost of running collectors and tail samplers. They are illustrative, not quotes, and should be re-checked before use. We also report a fourth, per-sample book, in which metrics are billed by ingested volume at the $0.50 per GB

listed for CloudWatch OpenTelemetry metrics [55], with an illustrative 100 bytes per sample. This book makes the scrape interval a cost lever.

Table 4 Steady-state volume and illustrative monthly cost (USD, 30 days) of single pipelines under three list-price books accessed 2026-10-05.

Pipeline	Volume per month	Datadog	Grafana	CloudWatch
Traces 100%	10,089 GB (9,137 spans/s)	1,009	5,549	3,531
Traces head 10%	1,009 GB	101	555	353
Traces head 1%	101 GB	10	55	35
Traces tail	394 GB (357 spans/s)	39	217	138
Logs 100%	7,137 GB	714	3,926	3,569
Logs uniform 1%	71 GB	7	39	36
Logs severity-aware 1%	111 GB	11	61	56
Metrics, tenant label dropped	944 series	47	6	283
Metrics, tenant cap 100	12,857 series	643	84	3,286
Metrics, tenant cap 500	59,256 series	2,963	385	7,926
Metrics, tenant uncapped	224,407 series	11,220	1,459	24,441

Span size 426 B [5]; log line sizes illustrative (Table 2). Series-based prices do not depend on scrape interval. Under the per-sample sensitivity book, the label-dropped metrics cost $12 per month at a 10 s scrape and $0.41 at 300 s.

The cost structure explains much of what follows. Span and log costs scale with traffic, while per-endpoint metric costs scale with the number of endpoints. Tenant-labelled metric costs scale with active endpoint–tenant pairs, of which our system has about 112,000. Under per-series pricing, the uncapped tenant label is the most expensive single item in the study: $11,220 per month under Datadog list prices, more than eleven times the cost of keeping every trace. The same tenant signal costs $39 per month as a span attribute under tail sampling.

We evaluated all 1,247 combinations of zero or one metric configuration (25 options), zero or one trace configuration (5 options) and zero or one log configuration (7 options). For each combination, an incident counts as detected if any member detects it, at the earliest member's TTD. We score combinations in two ways. **Coverage** is detection probability averaged over the five classes. **Timeliness** is the probability of detection within 5 minutes of the detectable onset, also averaged over classes. Of the 1,247 combinations, 1,049 are admissible under the false-alert criterion. Figure 4 shows them, and Table 5 lists the coverage frontier under Datadog list prices.

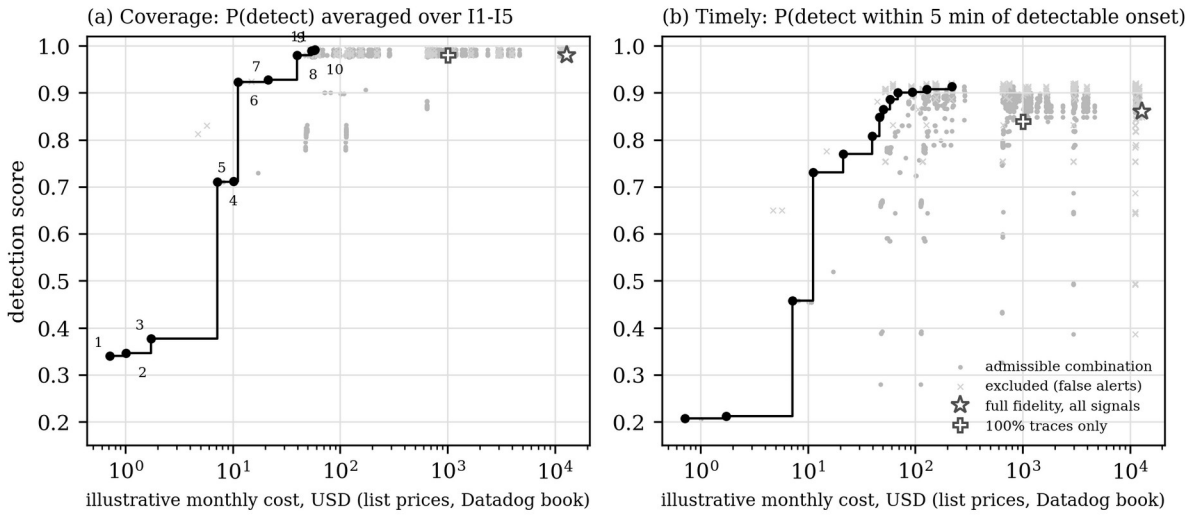


Fig. 4 Cost–detectability trade-off for 1,247 pipeline combinations under Datadog list prices accessed 2026-10-05 (log scale). Grey dots are admissible combinations; crosses are excluded for false alerts. The black step line is the Pareto frontier; numbers in (a) refer to Table 5. The star marks full fidelity in all signals (uncapped tenant-labelled 10 s metrics, 100% traces, 100% logs), and the cross marks 100% traces alone

Table 5 Coverage Pareto frontier under Datadog list prices, with the same combinations priced under the other books (USD per month). Coverage and timeliness are averaged over I1–I5.

#	Combination	Datadog	Grafana	CloudWatch	Coverage	Timely	False/day
1	Logs uniform 0.1%	0.7	3.9	3.6	0.340	0.207	1.0
2	Traces head 0.1%	1.0	5.5	3.5	0.346	0.206	0.9
3	Head 0.1% + logs uniform 0.1%	1.7	9.5	7.1	0.377	0.212	1.9
4	Logs uniform 1%	7.1	39	36	0.711	0.458	5.0
5	Traces head 1%	10.1	55	35	0.712	0.454	4.3
6	Logs severity-aware 1%	11.1	61	56	0.923	0.730	9.7
7	Head 1% + logs severity 1%	21.2	117	91	0.927	0.770	14.0
8	Traces tail	39.4	217	138	0.980	0.808	16.1
9	Metrics 300 s + logs uniform 1%	54.3	45	319	0.989	0.590	12.4
10	Metrics 300 s + head 1%	57.3	62	319	0.990	0.584	11.7
11	Metrics 300 s + logs severity 1%	58.3	67	339	0.991	0.784	17.1
–	Reference: full fidelity, all signals	12,943	10,933	31,541	0.980	0.860	30.5

Metrics in rows 9–11 carry no tenant label. False alerts of combinations are summed over members, an upper bound that assumes no de-duplication.

Five findings stand out. **First**, near-full coverage is cheap if the retained data are chosen well. Tail sampling alone matched the coverage of keeping everything (0.980), at $39 per month against $12,943 for full fidelity in all signals, about 0.3% of the cost. **Second**, the cheapest large step on the frontier comes from keeping failures, not from keeping a fraction. Severity-aware logs at 1% raised coverage from 0.711 (uniform 1%) to 0.923 for $4 more per month. **Third**, the only

coverage that tail sampling and severity-aware logs miss is long-window evidence for very sparse endpoints. That explains why the top of the frontier adds per-endpoint metrics with a 300 s scrape, whose 10-minute window gives the highest I2 detection. The gain is small (0.980 to 0.991) and comes at a cost in timeliness (0.784 against 0.808 for tail sampling alone). **Fourth**, under timeliness the frontier changes. It runs from tail sampling ($39, 0.808), to tail sampling plus severity-aware logs ($50.5, 0.865), to 120 s metrics plus severity-aware logs ($58, 0.886), to 60 s metrics with 10% head sampling and 10% uniform logs ($219, 0.913). Full fidelity in all signals scored 0.860, below several frontier points. Combining signals detects incidents sooner than any single signal at full fidelity. **Fifth**, tenant-labelled metrics never appeared on the frontier under any of the three vendor books. Under the per-sample sensitivity book, the cap of 500 appeared once on the coverage frontier, at $25.6 per month.

The frontier depends on the price structure. Under Grafana list prices, per-endpoint metrics are so cheap ($6 per month) that every coverage-frontier point above 0.8 includes them, and tail sampling drops off the coverage frontier because traces cost $0.55 per GB. Under CloudWatch prices, per-endpoint metrics cost $283 per month, and the frontier is built from traces and logs until $300. Under series-based pricing, the scrape interval is free on the bill. Under the per-sample book, a 10 s scrape costs 30 times as much as a 300 s scrape, and 60–120 s metrics dominate the low-cost end of the timeliness frontier. One recommendation holds under every book we examined: keep failures at full fidelity (by tail sampling or severity-aware logging), and carry high-cardinality identifiers on events, not on metric labels.

7 From simulation to deployment

The simulated pipelines correspond to settings that operators configure today. This section maps each one to OpenTelemetry and Prometheus configuration, expresses the alert rules in PromQL, and turns the models of Section 5 into a procedure for choosing settings. The configurations follow the component documentation as of October 2026 and are illustrative; the values shown are examples, and key names should be checked against the deployed release.

7.1 Configuration of each pipeline

Table 6 Mapping of the simulated pipelines to production configuration.

Pipeline	Where it is configured	Setting corresponding to the simulation	Notes
(a) Head sampling, rate s	OpenTelemetry SDK sampler, or Collector probabilistic sampler [56]	OTEL_TRACES_SAMPLER=parentbased_traceidratio with OTEL_TRACES_SAMPLER_ARG=s; or sampling_percentage = 100·s	Parent-based sampling keeps or drops whole traces, which matches the per-trace thinning of the simulation
(b) Tail sampling	Collector tail-sampling processor [51]	Policies: status_code ERROR; per root operation, an and policy of the operation name and a latency threshold; probabilistic 1%. decision_wait 30s; num_traces C	All spans of a trace must reach the same Collector instance; size C with Step 3 of Section 7.3
(c) Metrics, interval Δ	Prometheus scrape and rule configuration [34,36]	scrape_interval Δ; evaluation_interval 15s; a histogram bucket boundary at T_j; rules with for: 2m	Rule window max(5 min, 2Δ)
(d) Tenant cardinality cap	Instrumentation, SDK views or relabelling	Top-N allow-list mapping other tenants to "other"; label dropped by a view or a labeldrop relabelling rule	The SDK cardinality limit sends excess series to one overflow series marked otel.metric.overflow=true [57]; it keeps the first series seen, not the largest. sample_limit fails the whole scrape [34]
(e) Log sampling, rate r	Collector probabilistic sampler for logs, with a transform processor [56]	Uniform: sampling_percentage = 100·r. Severity-aware: a priority attribute set to 100 for records of severity ERROR or above, named in sampling_priority	Log support is alpha. Setting the priority also for slow requests removes the latency blind spot of Section 6.1

The listing below shows the tail-sampling and severity-aware logging pipelines as Collector configuration. The latency policy measures trace duration from the earliest span start to the latest span end [51], which matches the root duration used by our tail-sampler model. One latency policy is needed per root operation because thresholds differ by operation.

```
processors:
  tail_sampling:
    decision_wait: 30s
    num_traces: 50000              # C: size from the peak trace rate (Section 7.3, step 3)
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: slow-checkout         # one policy per root operation
        type: and
        and:
          and_sub_policy:
            - name: operation
              type: string_attribute
              string_attribute: {key: http.route, values: [/checkout]}
            - name: slow
              type: latency
              latency: {threshold_ms: 420}   # operation's baseline p99 (example)
      - name: baseline
        type: probabilistic
```

```
      probabilistic: {sampling_percentage: 1}
  transform/log-priority:
    log_statements:
      - context: log
        statements:
          - set(attributes["keep_pct"], 100) where severity_number >= SEVERITY_NUMBER_ERROR
  probabilistic_sampler/logs:
    sampling_percentage: 1        # r = 1%
    sampling_priority: keep_pct
```

7.2 Alert rules

For the metric pipelines, the error and latency rules of Table 2 are standard Prometheus alerting rules. The evidence guard becomes a second condition joined with *and*. The latency rule uses the histogram bucket at the SLO threshold, here 0.25 s as an example.

```
global:
  scrape_interval: 60s       # Δ: up to 120 s was classified safe in Table 3
  evaluation_interval: 15s
groups:
  - name: endpoint-slo
    rules:
      - alert: EndpointErrorRatio
        expr: |
          (  sum by (endpoint) (increase(requests_total{outcome="error"}[5m]))
           / sum by (endpoint) (increase(requests_total[5m])) > 0.02 )
          and
          sum by (endpoint) (increase(requests_total{outcome="error"}[5m])) >= 5
        for: 2m
      - alert: EndpointSlowFraction   # p99 above the SLO threshold T_j
        expr: |
          (  1 - sum by (endpoint) (increase(duration_seconds_bucket{le="0.25"}[5m]))
               / sum by (endpoint) (increase(duration_seconds_count[5m])) > 0.01 )
          and
          (  sum by (endpoint) (increase(duration_seconds_count[5m]))
           - sum by (endpoint) (increase(duration_seconds_bucket{le="0.25"}[5m])) >= 5 )
        for: 2m
```

Rules over sampled events, for example span counts produced from traces, must reweight the counts as in Section 4.2: by 1/*s* under head sampling, and by the policy-specific weights under tail sampling. A design we did not evaluate, but which follows from Section 6.3, is to take the denominator of a ratio from an unsampled request counter and only the numerator from failure-preserving events. That removes the noisy denominator that caused the extra false alerts of tail sampling and severity-aware logging.

7.3 A procedure for choosing reduction settings

The models of Section 5 need only quantities that operators already measure. The following procedure applies them; the thresholds in brackets are those of our study and should be replaced by the operator's own.

1. **List what must be detected.** For each service, record the smallest request rate λ that must be covered (from an unsampled request counter), the smallest failure ratio *e* to catch, and the rule window *w* and evidence guard *k*.
2. **Choose content-independent rates from the evidence.** For head-based trace sampling or uniform log sampling, require $x = \lambda\, e\, s\, w / k \geq 1$, that is, $s \geq k/(\lambda\, e\, w)$, for the smallest endpoint that must be covered (Equation 3). If the budget does not allow that rate, keep failures for that service with tail sampling or severity-aware logging instead.
3. **Size the tail sampler for peak load.** Set *C* so that *C/W_d* exceeds the peak trace arrival rate with margin. In our load model a buffer of 1.3 times mean occupancy, about 1.1 times the two-standard-deviation peak, lost no evidence, and losses grew abruptly below that (Section 6.4). Re-size when traffic grows.
4. **Keep latency evidence.** Give slow requests the same treatment as failures: a latency policy in the tail sampler, or a priority for slow requests in log sampling. Otherwise latency regressions are detected only as well as the content-independent rate allows.
5. **Choose the scrape interval by the price book.** With a 5-minute window, intervals up to 120 s were safe and 300 s was delayed for every class. Under per-series pricing the interval does not change the bill, so there is no saving from long intervals; under per-sample pricing, 60–120 s is a reasonable compromise.
6. **Decide where per-entity identifiers live.** List the entities, such as tenants, that need their own alerts. If they number more than a metric label can carry affordably, put the identifier on spans or logs and keep failures. If a cap is used, check Equation (6): an entity aggregated into "other" is detectable only if it carries more than φ of that bucket.
7. **Monitor the reductions themselves.** Alert on tail-sampler evictions and late-arriving spans, on failed scrapes and sample_limit violations, on overflow series, and on sampler settings drifting from their configured values. Each reduction fails silently otherwise.

8 Discussion

8.1 Which reductions are safe for which incidents

Applying the safe, delayed and lossy classification of Table 3 gives the following guidance. It is stated for our rules and parameters, but each item rests on a mechanism captured by the models of Section 5.

- **Head sampling and uniform log sampling are safe for high-volume signals and lossy for sparse ones.** At 10% they were safe for every class except low-traffic endpoints. At 1% they were lossy for I2, I3 and I4 and delayed for I5. Before choosing a rate, compute the evidence $x = \lambda\ e\ s\ w\ /\ k$ for the smallest endpoint and the smallest failure you need to catch, and keep $x \geq 1$.
- **Keeping all failures is the safest reduction for error incidents.** Tail sampling was safe for all five classes, adding 25–35 s of decision-and-export delay. Severity-aware logging at 1% was safe for every error class, but lossy for latency regressions because slow requests are not failures. Logs that carry latency need a slow-request rule as well as a severity rule.
- **Tail samplers need memory headroom above the peak-to-mean load ratio.** Below a headroom of about 1.2 the sampler failed abruptly. A sampler sized for average load loses evidence exactly when load peaks. Monitor the eviction rate, and re-size the buffer when traffic grows.
- **Scrape intervals up to 120 s were safe; 300 s was delayed for every class.** Under series-based pricing, the scrape interval does not change the bill, so long intervals bring delay without saving money. Under volume-based pricing it is a real cost lever, and 60–120 s is a reasonable compromise.
- **Metric cardinality caps are lossy for exactly the entities they drop.** A top-N cap preserves detection for the N largest tenants and removes it for everyone else, and Equation (6) shows why aggregation into "other" does not rescue them. If per-tenant detection matters, carry the tenant on spans or logs and keep failures. In our study that was cheaper by two orders of magnitude.
- **Watch the denominator.** Reductions that keep the numerator but thin the denominator (tail sampling, severity-aware logging) inflate estimated ratios for small series and create false alerts. Keep a larger probabilistic baseline, or require a minimum estimated volume, for per-tenant or per-endpoint rules on such pipelines.

8.2 Implications for system design

Three design implications follow. First, an alert's evidence requirement (k retained events in w) is part of the telemetry budget. Operators who sample should derive k and w together with the

sampling rate, not tune them separately. Second, a failure-preserving sampler moves cost from the vendor bill to the collector tier. Tail sampling needs stateful collectors with enough memory to hold every trace for the decision wait, and the OpenTelemetry documentation notes that this can require many nodes [4]. Our cost model does not price that compute, and an operator should add it. Third, Prometheus's *sample_limit* behaves very differently from the top-N cap we simulated. Exceeding it fails the whole scrape [34], so all of a target's metric-based alerts go silent at once. It should be paired with an alert on scrape failure. More generally, every reduction should be paired with an alert on the health of the reduction mechanism itself: eviction counts, dropped scrapes and sampler fall-back.

8.3 Threats to validity

Synthetic workload. All results come from a simulated system, not from production telemetry. Requests are independent given rates, failures are independent across requests, and errors are attributed to the endpoint where they originate. They do not propagate to callers, where they would create additional alert signals. Real incidents are often bursty, correlated across endpoints and accompanied by retries that change load. We hold traffic shares fixed during incidents, and we model latency only through the fraction of spans above a threshold.

Parameters chosen. The number of services, the rate distribution, the tenant Zipf exponent, the incident magnitudes, the rule thresholds, the window, the *for* duration and the evidence guard are our choices. They follow common practice where we could and are stated in Table 2, but they are not estimated from data. Different choices change the numbers. The models of Section 5 indicate how: through the evidence x for sampling, through Equations (4) and (5) for delay, and through Equation (6) for dilution. The benign-transient model determines the absolute false-alert rates, so only comparisons between pipelines should be read from them. Uniform log sampling and head trace sampling are statistically equivalent for our per-endpoint rules. A real system in which log volume per request varies by endpoint would break that equivalence.

No real production traces. The study makes no claim about any particular production system and was not validated against operational data. The tail-sampler model simplifies the real processor. We do not model spans co-retained because another span failed, trace-size limits, or late-arriving spans. Our eviction model assumes strict FIFO order.

Deployment mapping. The configurations in Section 7 were derived from the component documentation as of October 2026 and were not executed against a live Collector or Prometheus server in this study. Key names change between releases. The tail sampler's latency policy measures whole-trace duration; in production traces with asynchronous spans that duration can differ from the latency a user experiences.

Cost model. Prices are public list prices on one date and exclude discounts, allotments and infrastructure. Payload sizes for logs and metric samples are illustrative, and the span size is the 2010 Dapper average [5], which is smaller than many modern span encodings. Costs scale linearly with these assumptions. Frontier membership depends on the price book, as Section 6.6 shows, and we do not re-simulate at other traffic scales. At higher traffic, span and log costs grow while per-endpoint metric costs do not, and sampled pipelines gain evidence.

Statistical precision. With 500 trials per class, a detection probability near 0.5 has a 95% interval of about ±0.044. False-alert rates come from 10 simulated days, so rates below about 0.5 per day are imprecise.

9 Conclusion

Telemetry reduction is usually discussed as a cost problem with an unquantified risk. This paper quantifies the risk in a reproducible simulation and finds it highly uneven. Global incidents survived every reduction we tried. Failures on low-traffic endpoints were the first casualty of head sampling and uniform log sampling, and a single dimensionless quantity, the expected retained evidence per alert window, predicted when they would be missed. Reductions that keep every failure (tail sampling and severity-aware logging) preserved error detection at a few percent of the volume. Tail sampling also preserved latency detection, provided its buffer had headroom above peak load. Metric scrape intervals mattered less than expected up to 120 s. Cardinality caps were exactly as lossy as the entities they dropped, and the cheapest way to detect tenant-specific failures was to carry the tenant identifier on events. The simulator, analytical models and raw results are released so that operators can substitute their own traffic, rules and prices and re-derive the frontier for their systems. Section 7 maps each pipeline to OpenTelemetry and Prometheus configuration and turns the models into a procedure that operators can apply with measurements they already collect.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors. The work was conducted independently.

Conflict of interest. The author declares no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. This work was conducted independently, without funding, and uses only publicly available information.

Ethics approval and patient consent. Not applicable. The study uses only synthetic data generated by simulation and involved no human participants, personal data or animals.

Data availability statement. All data in this study are synthetic and are generated by the accompanying code. The simulator, the analysis and figure scripts, the raw per-trial outputs, the results file (results.json) and the figures are provided as Supporting Information S1 (paperF_code.zip). Running the scripts with the fixed seeds reproduces every reported result. Upon acceptance the material will be deposited in a public archival repository (Zenodo) with a DOI, and the link will be added to this statement. No proprietary, production or personal data were used.

Use of generative AI. During the preparation of this work the author used Claude (Anthropic) to assist with drafting and editing text and writing analysis code. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the article. This use is also described in Section 4.5.

Permission to reproduce material. Not applicable. All figures and tables are original.

Author contributions. Pujitha Sri Lakshmi Paladugu is the sole author. She conceived and designed the study, wrote the simulation and analysis code, performed the analysis, prepared the figures, and wrote and revised the manuscript. She read and approved the final manuscript.

References

1. Grafana Labs: Observability Survey 2025 (third annual survey, 1,255 responses; vendor-run). https://grafana.com/observability-survey/2025/ (2025). Accessed 5 October 2026
2. New Relic: 2023 Observability Forecast: challenges (vendor report). https://newrelic.com/resources/report/observability-forecast/2023/state-of-observability/challenges (2023). Accessed 5 October 2026
3. Cloud Native Computing Foundation: Cloud Native Observability MicroSurvey (186 responses). https://www.cncf.io/wp-content/uploads/2022/03/CNCF_Observability_MicroSurvey_030222.pdf (2022). Accessed 5 October 2026

4. OpenTelemetry Authors: Sampling. OpenTelemetry documentation (CC BY 4.0). https://opentelemetry.io/docs/concepts/sampling/. Accessed 5 October 2026
5. Sigelman, B.H., Barroso, L.A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., Shanbhag, C.: Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google (2010)
6. Prometheus Authors: Metric and label naming. Prometheus documentation. https://prometheus.io/docs/practices/naming/. Accessed 5 October 2026
7. Zseby, T., Molina, M., Duffield, N., Niccolini, S., Raspall, F.: Sampling and Filtering Techniques for IP Packet Selection. RFC 5475, IETF (2009). https://doi.org/10.17487/RFC5475
8. van Adrichem, N.L.M., Doerr, C., Kuipers, F.A.: OpenNetMon: network monitoring in OpenFlow software-defined networks. In: 2014 IEEE Network Operations and Management Symposium (NOMS), pp. 1–8 (2014). https://doi.org/10.1109/NOMS.2014.6838228
9. Chowdhury, S.R., Bari, M.F., Ahmed, R., Boutaba, R.: PayLess: a low cost network monitoring framework for software defined networks. In: 2014 IEEE Network Operations and Management Symposium (NOMS), pp. 1–9 (2014). https://doi.org/10.1109/NOMS.2014.6838227
10. Tangari, G., Tuncer, D., Charalambides, M., Qi, Y., Pavlou, G.: Self-adaptive decentralized monitoring in software-defined networks. IEEE Transactions on Network and Service Management 15(4), 1277–1291 (2018). https://doi.org/10.1109/TNSM.2018.2874813
11. Tang, S., Li, D., Niu, B., Peng, J., Zhu, Z.: Sel-INT: a runtime-programmable selective in-band network telemetry system. IEEE Transactions on Network and Service Management 17(2), 708–721 (2020). https://doi.org/10.1109/TNSM.2019.2953327
12. Chowdhury, S.R., Boutaba, R., François, J.: LINT: accuracy-adaptive and lightweight in-band network telemetry. In: 2021 IFIP/IEEE International Symposium on Integrated Network Management (IM) (2021). https://dl.ifip.org/db/conf/im/im2021/211967.pdf
13. Song, H., Qin, F., Martinez-Julia, P., Ciavaglia, L., Wang, A.: Network Telemetry Framework. RFC 9232, IETF (2022). https://doi.org/10.17487/RFC9232
14. Yan, B., Liu, Q., Shen, J., Liang, D., Liu, X.: Cost-effective and accurate flow statistics collection in OpenFlow-based SDN. International Journal of Network Management 32(4), e2197 (2022). https://doi.org/10.1002/nem.2197
15. Yahyaoui, H., Zhani, M.F., Bouachir, O., Aloqaily, M.: On minimizing flow monitoring costs in large-scale software-defined network networks. International Journal of Network Management 33(2), e2220 (2023). https://doi.org/10.1002/nem.2220
16. Mijumbi, R., Asthana, A., Koivunen, M., Haiyong, F., Zhu, Q.: Design, implementation, and evaluation of learning algorithms for dynamic real-time network monitoring. International Journal of Network Management 31(4), e2108 (2021). https://doi.org/10.1002/nem.2108
17. Ouyang, T., Yao, H., He, W., Mai, T., Wang, F.: Self-adaptive dynamic in-band network telemetry orchestration for balancing accuracy and stability. IEEE Transactions on Network and Service Management 22(2), 1514–1530 (2025). https://doi.org/10.1109/TNSM.2025.3530432
18. Yaseen, N., Arzani, B., Chintalapudi, K., Ranganathan, V., Frujeri, F., Hsieh, K., Berger, D.S., Liu, V., Kandula, S.: Towards a cost vs. quality sweet spot for monitoring networks. In: Proceedings of the 20th ACM Workshop on Hot Topics in Networks (HotNets '21), pp. 38–44 (2021). https://doi.org/10.1145/3484266.3487390
19. Kaldor, J., Mace, J., Bejda, M., Gao, E., Kuropatwa, W., O'Neill, J., Ong, K.W., Schaller, B., Shan, P., Viscomi, B., Venkataraman, V., Veeraraghavan, K., Song, Y.J.: Canopy: an end-to-end performance tracing

and analysis system. In: Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17), pp. 34–50 (2017). https://doi.org/10.1145/3132747.3132749

20. Las-Casas, P., Papakerashvili, G., Anand, V., Mace, J.: Sifter: scalable sampling for distributed traces, without feature engineering. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '19) (2019). https://doi.org/10.1145/3357223.3362736
21. He, S., Feng, B., Li, L., Zhang, X., Kang, Y., Lin, Q., Rajmohan, S., Zhang, D.: STEAM: observability-preserving trace sampling. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23), pp. 1750–1761 (2023). https://doi.org/10.1145/3611643.3613881
22. Chen, Z., Jiang, Z., Su, Y., Lyu, M.R., Zheng, Z.: TraceMesh: scalable and streaming sampling for distributed traces. In: 2024 IEEE 17th International Conference on Cloud Computing (CLOUD), pp. 54–65 (2024). https://doi.org/10.1109/CLOUD62652.2024.00016
23. Huang, Z., Chen, P., Yu, G., Chen, H., Zheng, Z.: Sieve: attention-based sampling of end-to-end trace data in distributed microservice systems. In: 2021 IEEE International Conference on Web Services (ICWS), pp. 436–446 (2021). https://doi.org/10.1109/ICWS53863.2021.00063
24. Zhang, L., Xie, Z., Anand, V., Vigfusson, Y., Mace, J.: The benefit of hindsight: tracing edge-cases in distributed systems. In: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23) (2023). https://www.usenix.org/conference/nsdi23/presentation/zhang-lei. Also available as arXiv:2202.05769
25. Huang, H., Chen, C., Chen, K., Chen, P., Yu, G., He, Z., Wang, Y., Zhang, H., Zhou, Q.: Mint: cost-efficient tracing with all requests collection via commonality and variability analysis. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), pp. 683–697 (2025). https://doi.org/10.1145/3669940.3707287
26. Ul Gias, A., Gao, Y., Sheldon, M., Perusquía, J.A., O'Brien, O., Casale, G.: SampleHST: efficient on-the-fly selection of distributed traces. In: NOMS 2023 – IEEE/IFIP Network Operations and Management Symposium, pp. 1–9 (2023). https://doi.org/10.1109/NOMS56928.2023.10154383
27. Gias, A.U., Gao, Y., Sheldon, M., Perusquía, J.A., O'Brien, O., Casale, G.: SampleHST-X: a point and collective anomaly-aware trace sampling pipeline with approximate half space trees. Journal of Network and Systems Management 32(3), 44 (2024). https://doi.org/10.1007/s10922-024-09818-8
28. Huang, H., Zhang, X., Chen, P., He, Z., Chen, Z., Yu, G., Chen, H., Sun, C.: TraStrainer: adaptive sampling for distributed traces with system runtime state. Proceedings of the ACM on Software Engineering 1(FSE), 473–493 (2024). https://doi.org/10.1145/3643748
29. Wu, Y., Yu, G., Jiang, Z., Li, Y., Lyu, M.R.: Trace Sampling 2.0: code knowledge enhanced span-level sampling for distributed tracing. arXiv preprint arXiv:2509.13852 (2025)
30. Zhu, Z., Jiang, Z., Yang, T., He, P.: UniSage: a unified and post-analysis-aware sampling for microservices. arXiv preprint arXiv:2509.26336, version 2 (2026)
31. Ertl, O.: Estimation from partially sampled distributed traces. arXiv preprint arXiv:2107.07703 (2021)
32. Pelkonen, T., Franklin, S., Teller, J., Cavallaro, P., Huang, Q., Meza, J., Veeraraghavan, K.: Gorilla: a fast, scalable, in-memory time series database. Proceedings of the VLDB Endowment 8(12), 1816–1827 (2015). https://doi.org/10.14778/2824032.2824078
33. Adams, C., Alonso, L., Atkin, B., Banning, J., Bhola, S., Buskens, R., Chen, M., Chen, X., Chung, Y., Jia, Q., Sakharov, N., Talbot, G., Tart, A., Taylor, N.: Monarch: Google's planet-scale in-memory time series database. Proceedings of the VLDB Endowment 13(12), 3181–3194 (2020). https://doi.org/10.14778/3181-3194
34. Prometheus Authors: Configuration. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/configuration/configuration/. Accessed 5 October 2026

35. Prometheus Authors: Recording rules. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/configuration/recording_rules/. Accessed 5 October 2026
36. Prometheus Authors: Alerting rules. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/. Accessed 5 October 2026
37. Trihinas, D., Pallis, G., Dikaiakos, M.D.: AdaM: an adaptive monitoring framework for sampling and filtering on IoT devices. In: 2015 IEEE International Conference on Big Data, pp. 717–726 (2015). https://doi.org/10.1109/BigData.2015.7363816
38. Huang, C.-K., Pierre, G.: AdapPF: self-adaptive scrape interval for monitoring in geo-distributed cluster federations. In: 2023 IEEE Symposium on Computers and Communications (ISCC), pp. 1–7 (2023). https://doi.org/10.1109/ISCC58397.2023.10218080
39. Fodor, B., Sonkoly, B.: Adaptive monitoring for cloud-native microservices. In: NOMS 2025 – IEEE/IFIP Network Operations and Management Symposium, pp. 1–5 (2025)
40. Wu, L., Tordsson, J., Elmroth, E., Kao, O.: MicroRCA: root cause localization of performance issues in microservices. In: NOMS 2020 – IEEE/IFIP Network Operations and Management Symposium, pp. 1–9 (2020). https://doi.org/10.1109/NOMS47738.2020.9110353
41. Yu, G., Chen, P., Li, P., Weng, T., Zheng, H., Deng, Y., Zheng, Z.: LogReducer: identify and reduce log hotspots in kernel on the fly. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1763–1775 (2023). https://doi.org/10.1109/ICSE48619.2023.00151
42. Liu, J., Zhu, J., He, S., He, P., Zheng, Z., Lyu, M.R.: Logzip: extracting hidden structures via iterative clustering for log compression. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 863–873 (2019). https://doi.org/10.1109/ASE.2019.00085
43. Zhang, L., Jia, T., Wang, K., Jia, M., Yang, Y., Li, Y.: Reducing events to augment log-based anomaly detection models: an empirical study. In: Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24) (2024). https://doi.org/10.1145/3674805.3695403
44. Karumuri, S., Solleza, F., Zdonik, S., Tatbul, N.: Towards observability data management at scale. ACM SIGMOD Record 49(4), 18–23 (2021). https://doi.org/10.1145/3456859.3456863
45. Brauckhoff, D., Tellenbach, B., Wagner, A., May, M., Lakhina, A.: Impact of packet sampling on anomaly detection metrics. In: Proceedings of the 6th ACM SIGCOMM Conference on Internet Measurement (IMC '06), pp. 159–164 (2006). https://doi.org/10.1145/1177080.1177101
46. Ishibashi, K., Kawahara, R., Mori, T., Kondoh, T., Asano, S.: Effect of sampling rate and monitoring granularity on anomaly detectability. In: 2007 IEEE Global Internet Symposium, pp. 25–30 (2007). https://doi.org/10.1109/GI.2007.4301426
47. Thurgood, S., Frame, J., Lenton, A., Quinito, C., Tolchanov, A., Trdin, N.: Alerting on SLOs. In: Beyer, B., Murphy, N.R., Rensin, D.K., Kawahara, K., Thorne, S. (eds.) The Site Reliability Workbook: Practical Ways to Implement SRE, Chap. 5. O'Reilly Media, Sebastopol, CA (2018). https://sre.google/workbook/alerting-on-slos/
48. Mai, J., Chuah, C.-N., Sridharan, A., Ye, T., Zang, H.: Is sampled data sufficient for anomaly detection? In: Proceedings of the 6th ACM SIGCOMM Conference on Internet Measurement (IMC '06), pp. 165–176 (2006). https://doi.org/10.1145/1177080.1177102
49. Harris, C.R., Millman, K.J., van der Walt, S.J., et al.: Array programming with NumPy. Nature 585, 357–362 (2020). https://doi.org/10.1038/s41586-020-2649-2
50. Virtanen, P., Gommers, R., Oliphant, T.E., et al.: SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 17, 261–272 (2020). https://doi.org/10.1038/s41592-019-0686-2

51. OpenTelemetry Authors: Tail Sampling Processor (README, opentelemetry-collector-contrib, main branch, stability: beta). https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/processor/tailsamplingprocessor/README.md. Accessed 5 October 2026
52. Horvitz, D.G., Thompson, D.J.: A generalization of sampling without replacement from a finite universe. Journal of the American Statistical Association 47(260), 663–685 (1952)
53. Datadog: Pricing list (US site; list prices). https://www.datadoghq.com/pricing/list/. Accessed 5 October 2026
54. Grafana Labs: Grafana Cloud pricing (Pro tier, "starts at" prices). https://grafana.com/pricing/. Accessed 5 October 2026
55. Amazon Web Services: Amazon CloudWatch pricing (worked examples, US East (N. Virginia)). https://aws.amazon.com/cloudwatch/pricing/. Accessed 5 October 2026
56. OpenTelemetry Authors: Probabilistic Sampling Processor (README, opentelemetry-collector-contrib, main branch; stability: beta for traces, alpha for logs). https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/processor/probabilisticsamplerprocessor/README.md. Accessed 7 October 2026
57. OpenTelemetry Authors: Metrics SDK specification, Cardinality limits (CC BY 4.0). https://opentelemetry.io/docs/specs/otel/metrics/sdk/. Accessed 7 October 2026